

Rule semantics

$$\langle \cdot \rangle : \mathcal{U}_T \rightarrow \mathcal{U}_T, \quad \llbracket r \rrbracket : \mathcal{F}_T \rightarrow \mathcal{F}_T$$

$$\frac{t_1 :: u, \dots, t_n :: u \text{ are all tagged-tuples in } U_T \quad t = \bigoplus_{i=1}^n t_i \quad \text{discard}(t) = \text{false}}{t :: u \in \langle U_T \rangle} \text{(NORMALIZE)}$$

$$\frac{t^{\text{old}} :: u \in \llbracket p \rrbracket(F_T) \quad \langle \llbracket e \rrbracket(F_T) \rangle \not\models u}{t^{\text{old}} :: p(u) \in \llbracket p \leftarrow e \rrbracket(F_T)} \text{(RULE-1)} \quad \frac{t^{\text{new}} :: u \in \langle \llbracket e \rrbracket(F_T) \rangle \quad \llbracket p \rrbracket(F_T) \not\models u}{t^{\text{new}} :: p(u) \in \llbracket p \leftarrow e \rrbracket(F_T)} \text{(RULE-2)}$$

$$\frac{t^{\text{old}} :: u \in \llbracket p \rrbracket(F_T) \quad t^{\text{new}} :: u \in \langle \llbracket e \rrbracket(F_T) \rangle}{(t^{\text{old}} \oplus t^{\text{new}}) :: p(u) \in \llbracket p \leftarrow e \rrbracket(F_T)} \text{(RULE-3)}$$

Stratum and Program semantics

$$\text{lfp}^\circ : (\mathcal{F}_T \rightarrow \mathcal{F}_T) \rightarrow (\mathcal{F}_T \rightarrow \mathcal{F}_T), \quad \llbracket s \rrbracket, \llbracket \bar{s} \rrbracket : \mathcal{F}_T \rightarrow \mathcal{F}_T$$

- (SATURATION) $F_T^{\text{old}} \triangleq F_T^{\text{new}}$ iff $\forall t^{\text{new}} :: p(u) \in F_T^{\text{new}}, \exists t^{\text{old}} :: p(u) \in F_T^{\text{old}}$ such that $t^{\text{old}} \oplus t^{\text{new}}$
- (FIXPOINT) $\text{lfp}^\circ(h) = h \circ \dots \circ h = h^n$ if there exists a minimum $n > 0$, such that $h^n(F_T) \triangleq h^{n+1}(F_T)$
- (STRATUM) $\llbracket s \rrbracket = \text{lfp}^\circ(\lambda F_T. (F_T - \bigcup_{p \in P_s} F_T[p]) \cup (\bigcup_{r \in S} \llbracket r \rrbracket(F_T)))$
- (PROGRAM) $\llbracket \bar{s} \rrbracket = \llbracket s_n \rrbracket \circ \dots \circ \llbracket s_1 \rrbracket$, where $\bar{s} = s_1; \dots; s_n$.

Fig. 24. Operational semantics of SCLRAM (Cont.)

Operation	Algorithm	Time Complexity
$t_1 \oplus t_2$	$\max(t_1, t_2)$	$O(1)$
$t_1 \otimes t_2$	$\min(t_1, t_2)$	$O(1)$
$\ominus t$	$\hat{1} - t$	$O(1)$
$t_1 \ominus t_2$	$t_1 == t_2$	$O(1)$
$\rho(t)$	t	$O(1)$

Table 6. diff-max-min-prob provenance structure run time analysis

Operation	Algorithm	Time Complexity
$t_1 \oplus t_2$	$\text{clamp}(t_1 + t_2)$	$O(n)$
$t_1 \otimes t_2$	$t_1 \cdot t_2$	$O(n)$
$\ominus t$	$\hat{1} - t$	$O(n)$
$t_1 \ominus t_2$	true	$O(1)$
$\rho(t)$	t	$O(1)$

Table 7. diff-add-mult-prob provenance structure run time analysis

Operation	Algorithm	Time Complexity
$t_1 \oplus t_2$	$\text{top}_k(t_1 \cup t_2)$	$O(nk)$
$t_1 \otimes t_2$	$\text{top}_k(\{ \eta \mid (\eta_1, \eta_2) \in t_1 \times t_2, \eta = \eta_1 \cup \eta_2, \eta \text{ no conflict} \})$	$O(n^2 k^2)$
$\ominus t$	$\text{top}_k(\text{cnf2dnf}(\{ \{ \neg v \mid v \in \eta \} \mid \eta \in t \})$	$O(2^n)$
$t_1 \ominus t_2$	$t_1 == t_2$	$O(nk)$
$\rho(t)$	$\text{WMC}(t, \Gamma)$	$O(2^n)$

Table 8. diff-top-k-proofs provenance structure run time analysis. We assume the term number for each tag is of $O(n)$, and the number of clauses is of $O(k)$. Note that we show the time complexity for the naive implementation of cnf2dnf and WMC algorithms.**B.4.3 diff-top-k-proofs.**

Runtime Analysis. We showcase the time complexity for the diff-top-k-proofs provenance structure in Table 8. Note that we show the runtime analysis for the naive implementation of the cnf2dnf function and weighted model counting process. We leave the optimizations for future work.

B.4.4 diff-top-k-proofs-me.

Support for Mutually Exclusive Facts. We keep the original diff-top-k-proofs and extend it to support mutual exclusive tags (hence the name -me). Most of the operations remains, and we start by giving it a new input tag space:

$$I_{\text{dtkp-me}} = [0, 1] \times \text{Option}\langle \mathbb{N} \rangle.$$

The probability remains the same, and the second part is an optional identifier of one mutual exclusion. Not providing that means the fact does not belong to a mutual exclusion. If presented, we denote such a tag

$$t_i = (r_i, \text{Some}(\mathbf{me}_i))$$

Example B.1. Consider the sum2 task, we have two distributions specified by

```
rel digit = {0.90::(A, 0); 0.01::(A, 1); ...; 0.02::(A, 9)}
rel digit = {0.01::(B, 0); 0.91::(B, 1); ...; 0.01::(B, 9)}
```

Then we would have the input facts being

$$\llbracket \text{digit} \rrbracket (F_{\text{Option} \langle \cdot \rangle}) = \left\{ \begin{array}{l} (0.90, \text{Some}(1)) \quad :: \quad (A, 0), \\ (0.01, \text{Some}(1)) \quad :: \quad (A, 1), \\ \vdots \\ (0.02, \text{Some}(1)) \quad :: \quad (A, 9), \\ (0.01, \text{Some}(2)) \quad :: \quad (B, 0), \\ (0.91, \text{Some}(2)) \quad :: \quad (B, 1), \\ \vdots \\ (0.01, \text{Some}(2)) \quad :: \quad (B, 9) \end{array} \right\}.$$

There are two **me**-s, namely 1 and 2, suggesting there are two mutual exclusions. The first 10 facts for digit *A* belong to mutual exclusion 1, and the next 10 for digit *B* belong to mutual exclusion 2.

Putting it in action, two facts f_i and f_j with the same **me** ($\mathbf{me}_i = \mathbf{me}_j$) cannot appear together in the same proof. This is enforced inside of the conflict check during dtkp 's $\wedge_k$ and $\neg_k$ operations. If the proof violates the mutual exclusive assumption, it will be deemed invalid proof and discarded. For example, executing the following rule

```
rel not_possible() = digit(A, x), digit(A, y), x != y
```

shall yield no valid output due to this mechanism, i.e., the internal tag associated with `not_possible()` is going to be $\emptyset$ and be early-removed from the computation.

C EVALUATION

C.1 MNIST-R

Experimental Setup. The total MNIST[Lecun et al. 1998] dataset contains 60K handwritten digits. According to the number of digits used in each task, we further split the MNIST dataset into downstream task data points. For example, we have 30K training pairs for sum-2, 20K training pairs for sum-3, and 15K training pairs for sum-4. As the counting task consumes 8 images per datapoint, we have 7.5K training pairs for the counting tasks. The test and training splits follow the original MNIST task setup. Our neural module consists of a 2-layer CNN connected by a 2-layer MLP, with a hidden layer size of 1024. We trained the Scallop module 10 epochs on the training dataset, the learning rate is set to 0.001, and the batch size is 64, with a BCE-loss closing the training loop.

Scallop code. We present the Scallop code that is used in the MNIST-R benchmark in Fig. 25.

C.2 Hand Written Formula

```

type digit_1(u32), digit_2(u32)
rel sum_2(a + b) = digit_1(a), digit_2(b)

```

(a) Scallop code for MNIST sum-2.

```

type digit_1(u32), digit_2(u32), digit_3(u32)
rel sum_3(a + b + c) = digit_1(a), digit_2(b), digit_3(c)

```

(b) Scallop code for MNIST sum-3.

```

type digit_1(u32), digit_2(u32), digit_3(u32), digit_4(u32)
rel sum_4(a + b + c + d) = digit_1(a), digit_2(b), digit_3(c), digit_4(d)

```

(c) Scallop code for MNIST sum-4.

```

type digit_1(u32), digit_2(u32)
rel less_than(a < b) = digit_1(a), digit_2(b)

```

(d) Scallop code for MNIST less-than.

```

type digit(u32)
rel not_3_or_4() = not digit(3) and not digit(4)

```

(e) Scallop code for MNIST not-3-or-4.

```

type digit(digit_id: u32, digit_value: u32)
rel count_3(x) :- x = count(o: digit(o, 3))

```

(f) Scallop code for MNIST count-3.

```

type digit(digit_id: u32, digit_value: u32)
rel count_3_or_4(x) = x = count(o: digit(o, 3) or digit(o, 4))

```

(g) Scallop code for MNIST count-3-or-4.

Fig. 25. Scallop code for MNIST-R.

Experimental Setup. The HWF [Li et al. 2020] dataset takes in a sequence of images, where each image can be either a digit or an arithmetic symbol, and aims to calculate the outcome of the formula. We apply the same CNN-based symbol recognition network as for the MNIST-based tasks, with the only difference being that the model now performs a 14-way classification. We then generate a probabilistic database containing two relations: 1) `length(usize)`, a non-probabilistic relation storing the length of the formula, and 2) `symbol(id: usize,  $\sigma$ : String)`, a probabilistic relation mapping index IDs to symbols $\sigma \in \Sigma$. For the reasoning component, we write a formula parser in Scallop (as shown in 26). Additionally, Scallop is used to perform calculations based on the parsed abstract syntax tree (AST) to obtain the final rational number output. The training loop is closed by a BCE loss at the end, forming an end-to-end training pipeline where gradients can back-propagate all the way from the output to the symbol recognition network. To prune the enormous output domain if we enumerate every possible parse tree, we sample only a subset of symbol predictions to be sent to Scallop. There are 10K images answer pairs for the training dataset. We set the batch size to 16, the learning rate to 0.0001, and the training epochs to 100. To enhance the learning efficiency, we adopt a sampling strategy, which only preserves the 7 most likely classification result for each image classification task, and sends it to Scallop.